D:\Downloads\main (2).cpp

```cpp
#include <iostream>
#include <map>
#include <list>
#include <unordered_map>
#include <algorithm>
#include <iomanip>
using namespace std;

using Price = long long;
constexpr Price SCALE = 10000;
constexpr Price toPrice(double p) { return static_cast<Price>(p * SCALE + 0.5); }
constexpr double toDouble(Price p) { return static_cast<double>(p) / SCALE; }

struct Order {
    int id; Price p; int total; int filled = 0;
    int remains() const { return total - filled; }
};
struct IndexInfo { Price p; bool isBid; list<Order>::iterator it; };
class OrderBook {
    map<Price, list<Order>, greater<Price>> bids; // 买：降序
    map<Price, list<Order>> asks;                 // 卖：升序
    unordered_map<int, IndexInfo> index;

public:
    void insert(int id, double pRaw, char dir, int qty) {
        if (index.count(id)) return;
        Price p = toPrice(pRaw);
        bool isBid = (dir == 'B');
        auto doInsert = [&](auto& book) {
            book[p].push_back({id, p, qty});
            index[id] = {p, isBid, std::prev(book[p].end())};
        };
        if (isBid) doInsert(bids); else doInsert(asks);
        match();
    }
    void cancel(int id) {
        if (auto it = index.find(id); it != index.end()) {
            auto [p, isBid, listIt] = it->second;
            auto doErase = [&](auto& book) {
                auto& list = book[p];
                list.erase(listIt);
                if (list.empty()) book.erase(p);
            };
            if (isBid) doErase(bids); else doErase(asks);
            index.erase(it);
        }
    }
    void match() {
        while (!bids.empty() && !asks.empty()) {
            auto bit = bids.begin();
            auto ait = asks.begin();
```

```
            if (bit->first < ait->first) break; // 买价 < 卖价，无法成交

            Order& bid = bit->second.front();
            Order& ask = ait->second.front();
            int qty = min(bid.remains(), ask.remains());
            bid.filled += qty;
            ask.filled += qty;
            cout << ">>> 成交: 数量 " << qty << " @ 价格 " << toDouble(ask.p) << endl;

            auto clean = [&](auto& mapIt, auto& map) {
                if (mapIt->second.front().remains() == 0) {
                    index.erase(mapIt->second.front().id);
                    mapIt->second.pop_front();
                    if (mapIt->second.empty()) map.erase(mapIt);
                }
            };
            clean(bit, bids);
            clean(ait, asks);
        }
    }
    Order query(int id) const {
        if (auto it = index.find(id); it != index.end()) return *it->second.it;
        return {-1};
    }
    void show() const {
        auto print = [](const auto& book, string name) {
            cout << name << ": ";
            for (const auto& [p, list] : book) {
                int vol = 0;
                for (const auto& o : list) vol += o.remains();
                cout << "[" << toDouble(p) << ":" << vol << "] ";
            }cout << endl;
        };
        print(asks, "Sell");
        print(bids, "Buy ");
    }
};
int main() {
    OrderBook ob;
    ob.insert(1, 100.0, 'B', 10);
    ob.insert(2, 99.0, 'S', 5);
    ob.show(); // 预期：买单剩5个，卖单空
    Order o = ob.query(1);
    if (o.id != -1) cout << "ID: " << o.id << " 剩余: " << o.remains() << endl;
    return 0;
}
```